

Documentação do Projeto: Sistema de Gestão e Fidelização Ótica

1. Resumo Executivo

O projeto visa a transição do acompanhamento manual para uma plataforma digital automatizada, permitindo escala sem perder a personalização. O foco central é a **fidelização e humanização**, replicando digitalmente o cuidado presencial oferecido pela proprietária.

2. Definição do Ecossistema

2.1 Público-Alvo:

- **Ator:** Plataforma
- **Foco de Atuação:** Administrador, Gestão estratégica, aprovações e controle operacional.
- **Cliente:** Aplicativo Mobile (Android), Agendamentos, histórico de saúde visual e gamificação.

2.2 Fluxos de Experiência

- **Fluxo do Cliente:** Focado em autonomia (agendamentos), transparência (status da confecção) e engajamento (pontuação e indicações).
 - **Fluxo do Administrador:** Focado em controle proativo (gestão de agenda) e manutenção do relacionamento (edição de mensagens automáticas).
-

3. Estratégia de Comunicação e Fidelização

3.1 Régua de Relacionamento (Notificações)

O sistema opera com gatilhos temporais automáticos, permitindo revisão humana antes do disparo.

- **15 Dias:** Pesquisa de satisfação e adaptação inicial.
- **30 Dias:** Suporte técnico e reforço de orientações de uso.

- **03 Meses:** Check-up de manutenção (limpeza e ajustes gratuitos).
- **06 Meses:** Revisão intermediária de conforto.
- **09 Meses:** Segundo lembrete de ajuste preventivo.
- **12 Meses:** Alerta de renovação de grau (Ciclo de recompra).
- **Aniversários:** Mensagens puramente relacionais (sem viés comercial).

3.2 Gamificação (Sistema de Scores)

Ações que geram pontos para conversão em benefícios futuros:

1. **Agendamento de Consultas/Manutenções:** Recompensa Média.
2. **Avaliação de Serviços:** Recompensa Baixa.
3. **Indicação de Novos Clientes:** Recompensa Alta (Validada após conversão/uso do link).

4. Especificações do Sistema

4.1 Requisitos Funcionais (RF)

Módulo Administrativo (Web)

- [] **RF01:** Gestão completa de usuários (CRUD Clientes e Administradores).
- [] **RF02:** Autenticação segura com níveis de acesso.
- [] **RF03:** Gestão de Agenda (Disponibilidade para consultas/limpezas).
- [] **RF04:** Visualização de Agendamentos.
- [] **RF05:** Painel de Status de Produção (Acompanhamento da confecção).
- [] **RF06:** Central de Ocorrências (Leitura e resposta de feedbacks/reclamações).
- [] **RF07:** Editor de Templates (Personalização das mensagens da régua de relacionamento).

Módulo do Cliente (Mobile)

- [] **RF08:** Solicitação de serviços e agendamentos via calendário.
- [] **RF09:** Visualização de status do pedido em tempo real.
- [] **RF10:** Registro de ocorrências e sugestões.
- [] **RF11:** Carteira de Pontos (Visualização do Score).
- [] **RF12:** Módulo de indicação "Indique um Amigo".

4.2 Requisitos Não Funcionais (RNF) e Regras de Implementação

Segurança: Todos os dados sensíveis devem ser criptografados. A autenticação utilizará:

- **Spring Security + JWT**
 - **ID**
 - **Categoria (Perfis admin ou cliente)**
-

Descrição Técnica / Regra de Negócio

- **RNF01:**
 - **Banco de Dados:** Uso de **PostgreSQL** para dados relacionais e integração com **Gmail** para mensagens.
 - **RNF02:**
 - **Processamento:** Aplicação de descontos automática ao atingir a margem de pontos definida.
 - **RNF03:**
 - **Interface:** O Score do cliente deve estar visível no Header (canto superior direito).
 - **RNF04:**
 - **Performance:** Notificações Push devem ser disparadas com baixa latência.
-

5. Lógicas de Implementação

- **5.1 Administrador**
 - **5.1.1 - Cadastrar Usuários Clientes:**
 - Preencher todos os campos.
 - Coletar dados: Nome, Data de Nascimento, Telefone, Email, Senha.
 - Tratar dados de input.
 - inserir no banco de dados relacional (POSTGRES).
 - **5.1.2 - Fazer autenticação via Login:**
 - Preencher todos os campos.
 - Coletar dados: Login e Senha.
 - Criptografar dados via Spring-Security e JWT.

- Tratar dados criptografados por Spring-Security → JWT.
- Prosseguir para a próxima atividade.
- **5.1.3 - Gestão de Consultas:**
 - Definir dia do mês e hora para a consulta.
 - Tratar data e hora para selecionar o tipo de consulta.
- **5.1.4 - Visualização de Agendamentos:**
 - Retornar agendamento com data e horário.
- **5.1.5 - Painel de Status de Confeção:**
 - Definir Cliente.
 - Notificar Compra.
 - Definir Status.
 - Confirmar.
- **5.1.6 - Acesso a histórico de ocorrências:**
 - *Opção: Conferir avisos.*

Histórico de ocorrências:

 - Leitura de mensagens
 - Apagar mensagem
 - Apagar tudo
 - Apagar automaticamente

Reclamações:

 - Leitura de mensagens
 - Apagar mensagem
 - Responder
 - Apagar automaticamente
- **5.1.7 - Editor de mensagens pré-programadas:**
 - *Configurações → Mensagens Automáticas;*
 - Selecionar o tipo de mensagem automática: Lembrete, Notificação de Compra, Aniversário;

5.2 Usuário (Cliente)

- **5.2.1 - Login e autenticação de usuário:**
 - Preencher todos os campos de login e senha.
 - criptografar os dados recebidos.
 - comparar os dados cadastrados no banco de dados com os recebidos.
 - Prosseguir para a próxima página.
 - **5.2.2 - Solicitação de serviços/agendamentos:**
 - Selecionar o tipo de serviço.
 - selecionar data e hora disponível.
 - confirmar solicitação.
 - **5.2.3 - Pannel de pedido (confeção).**
 - Acompanhar pedido.
 - **5.2.4 - Registro de ocorrências e reclamações.**
 - Selecionar o tipo de serviço: Reclamações, Registrar Ocorrências.
 - Registrar.
 - Enviar registro.
 - **5.2.5 - Consulta de pontos.**
 - Visualizar Score presente no canto superior esquerdo da tela.
 - **5.2.6 - Compartilhar App com amigos.**
 - Selecionar para onde quer compartilhar.
 - Selecionar contato.
-

5.3 Funções Sistemáticas

- **5.3.1 - Aplicação de Desconto via Score:**
 - Buscar e retornar o cliente.
 - Retornar o total da pontuação referente ao cliente.
 - Verificar se a pontuação já chegou à margem necessária para receber o desconto.
 - Conceder desconto.

- Notificar usuário (cliente) informando o valor do desconto e que será aplicado na próxima consulta.
 - Notificar usuário (administrador) do desconto concedido.
 - **5.3.2 - Lembretes a cada data específica:**
 - Emitir mensagem configurada pelo usuário Administrador a cada evento específico para o cliente.
-

5.4 Respostas de execuções

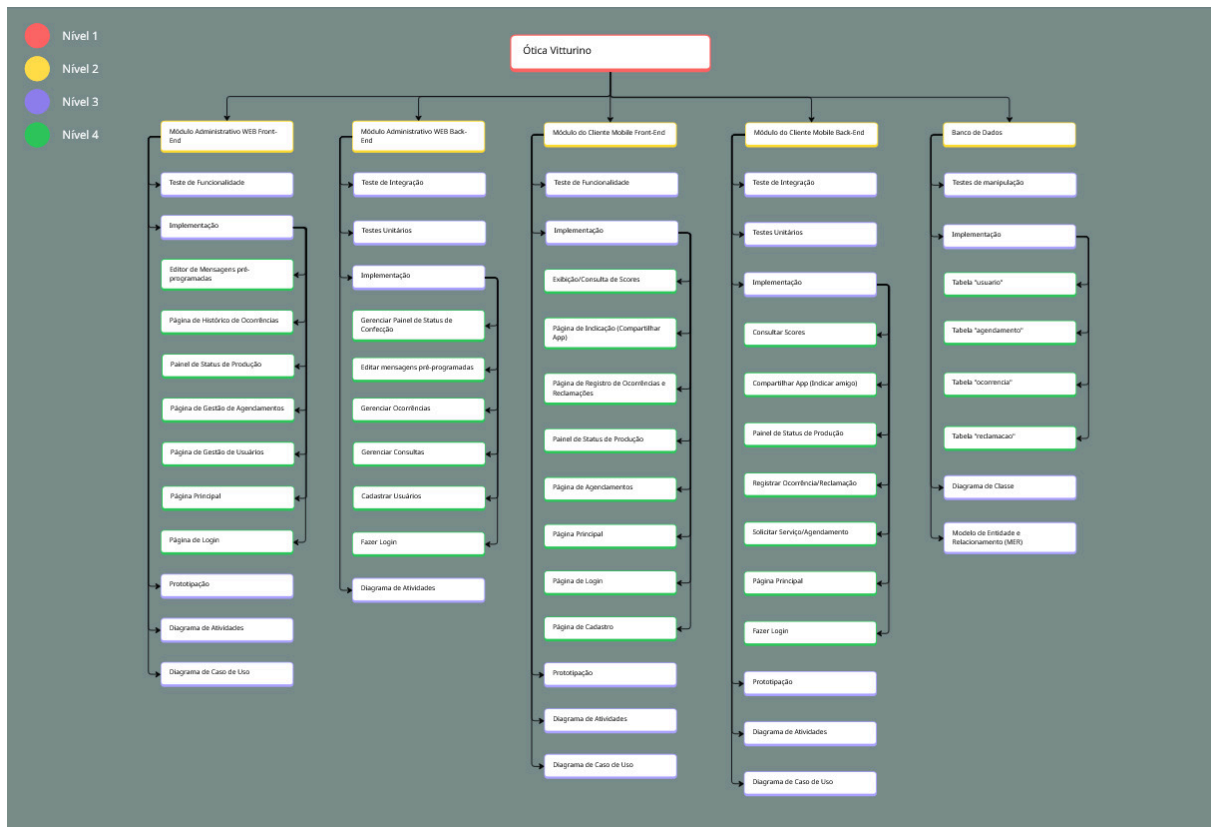
- **5.4.1 - Segurança:** Criptografia de dados sensíveis (Todos os dados de Cliente e Administrador).
-



6. Stack Tecnológica

- **Back-End:** Java (Spring Boot).
- **Front-End Web:** React.js.
- **Mobile:** React Native.
- **Infraestrutura:** Hostinger (ou similar a definir conforme escalabilidade).
- **Arquitetura de Software:** MVC (*Model View Controller*).

EAP (Estrutura Analítica do Projeto)

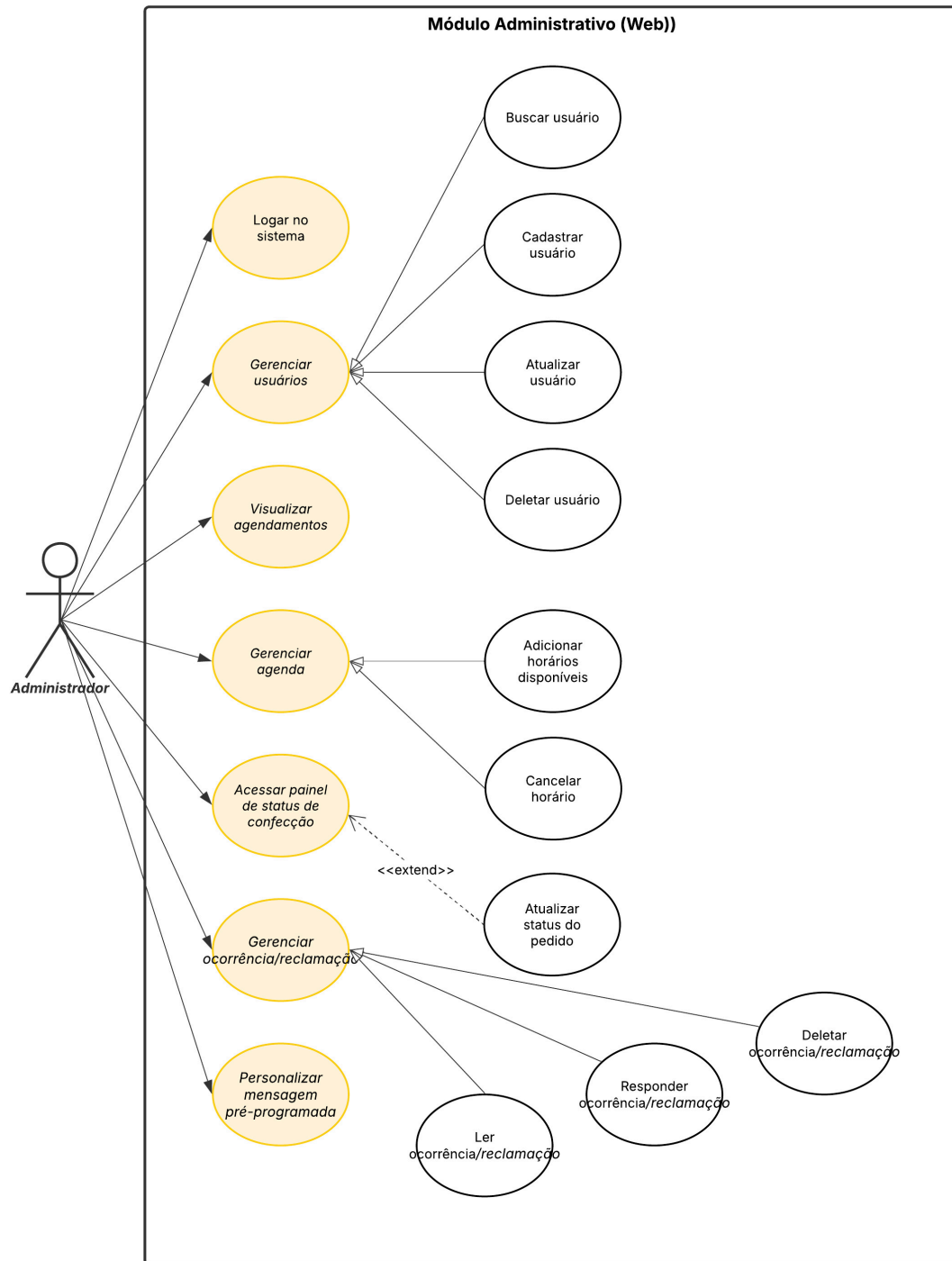


Diagramas de Caso de Uso

Módulo Administrativo (Web):

Ótica Vitturino - Módulo Administrativo (Web)

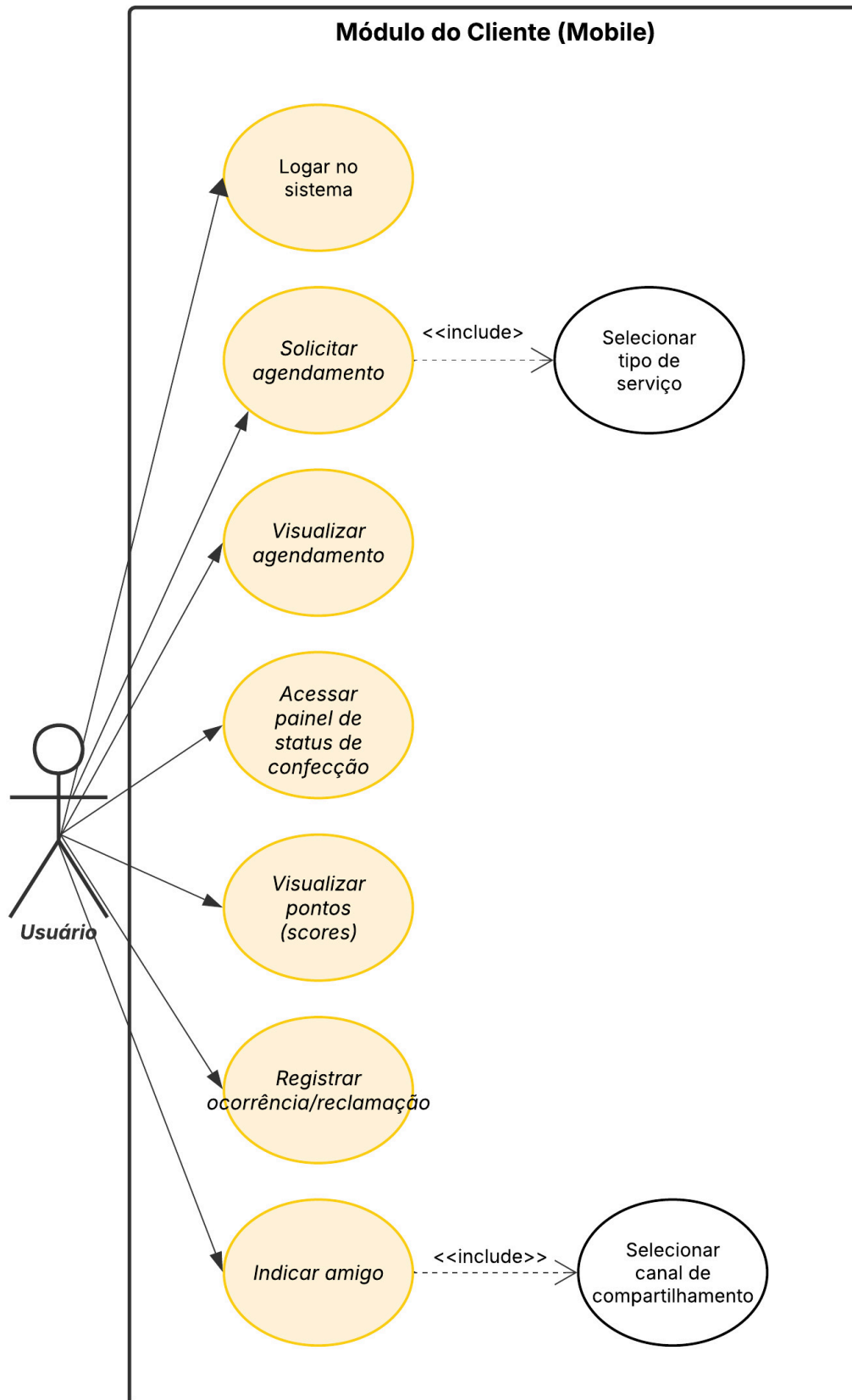
Caio Vasconcelos | March 9, 2026



Módulo do Cliente (Mobile):

Ótica Vitturino - Módulo do Cliente (Mobile)

Caio Vasconcelos | March 9, 2026

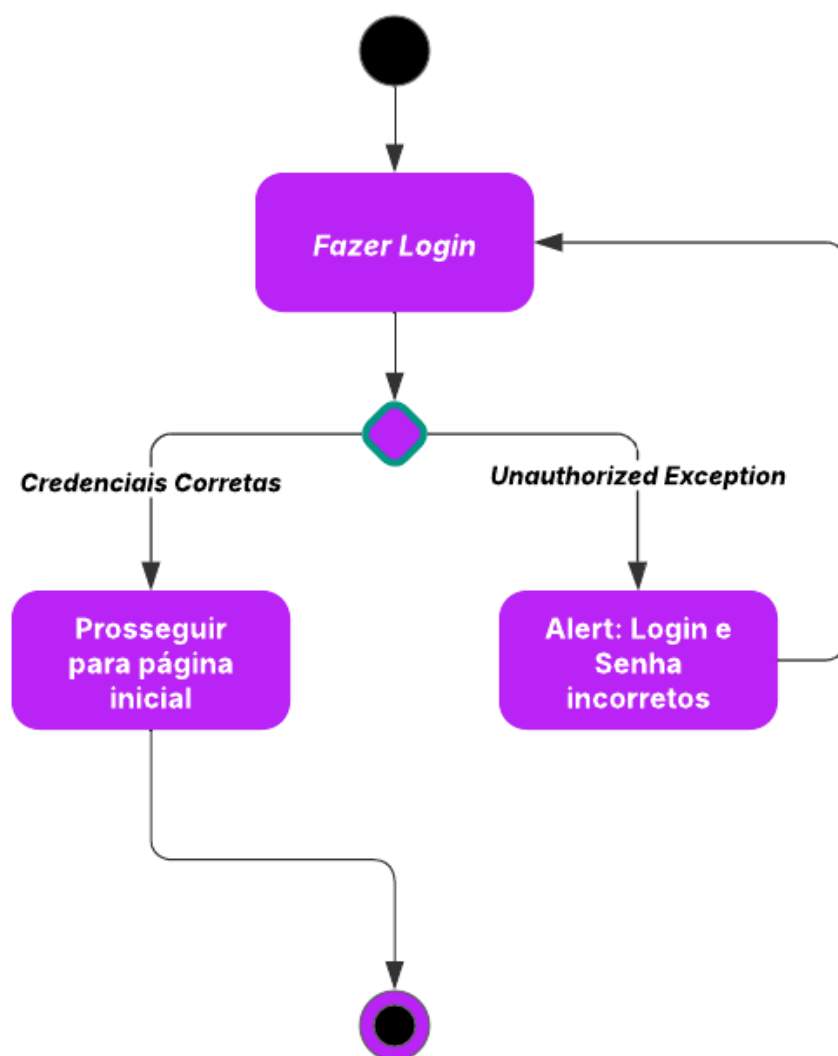


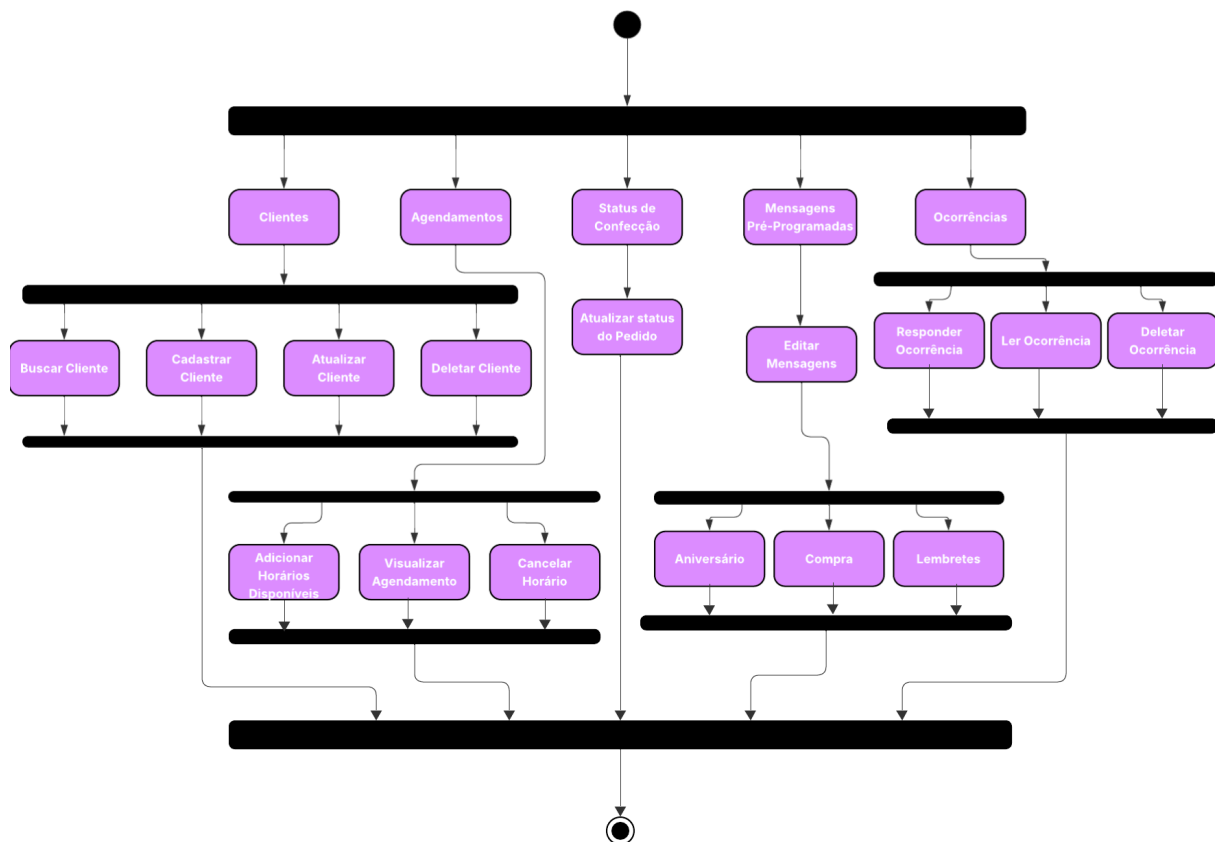
Diagramas de Atividades

Sistema Web → Administrador

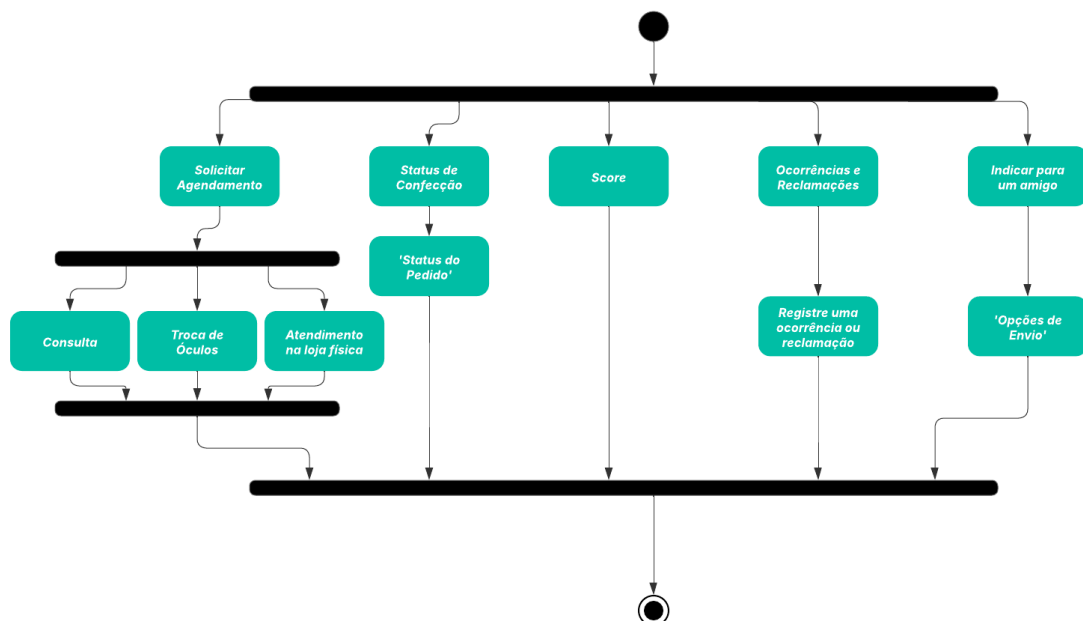
Diagrama de atividade

Daniel Silva | March 10, 2026



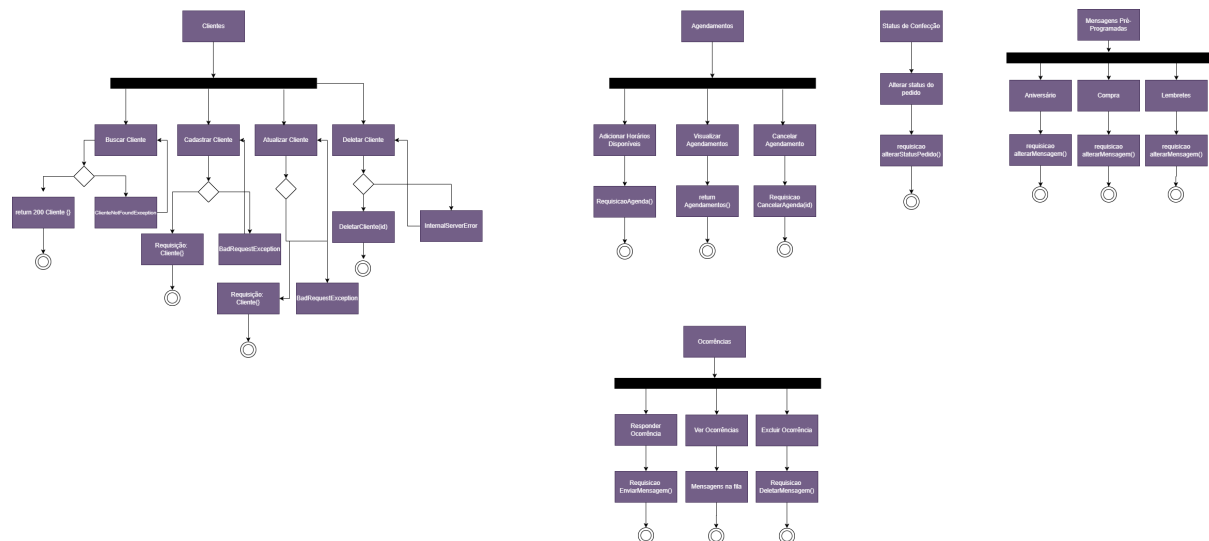


Sistema Mobile → Cliente

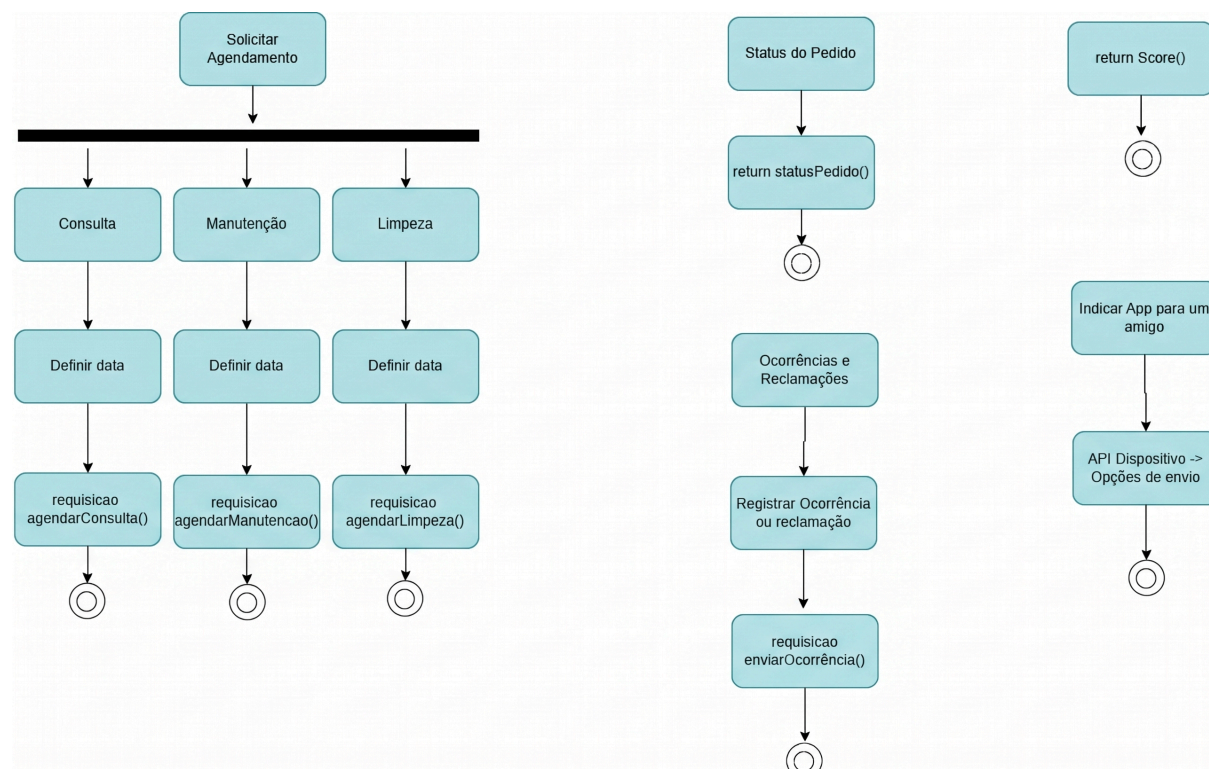


Fluxo de implementação

Sistema Web → Administrador



Sistema Mobile → Cliente





Arquivo .sql

Banco de Dados Ótica Vitturino.sql



Modelo Entidade-Relacionamento (MER)

1. Entidades, Atributos e Chaves:

- **usuario**

- `id_usuario` (Chave Primária)
- `nome`
- `email`
- `senha`
- `perfil`

- **cliente**

- `id_cliente` (Chave Primária)
- `nome`
- `email`
- `senha`

- **agendamento**

- `id_agendamento` (Chave Primária)
- `tipo_agendamento` (Ex: Consulta, manutenção, limpeza)
- `data_agendamento`
- `status`
- `id_usuario` (**Chave Estrangeira**) → *Adicionado para saber qual usuário fez o agendamento.*

- **ocorrencia** (Registros internos feitos pelos admins)

- `id_ocorrencia` (Chave Primária)
- `descricao`
- `data_ocorrencia`
- `resposta_admin`
- `data_resposta`
- `id_usuario` (**Chave Estrangeira**) → Adicionado para saber qual usuário registrou o problema.
- `id_admin` (**Chave Estrangeira**)
- **pedido** (Armações, lentes, óculos de sol)
 - `id_pedido` (Chave Primária)
 - `nome_pedido`
 - `status` (Ex: Pedido realizado, Em andamento, Pronto)
 - `id_usuario` (**Chave Estrangeira**) → Adicionado para referenciar o administrador (passível de remoção dependendo do cenário).

2. Regras de Relacionamento e Cardinalidade:

Relação: Cliente ↔ Agendamento (Ação: "cliente agenda")

- **Leitura do diagrama:** `1..1` de ambos os lados.
- **Regra atual:** Um `cliente` pode realizar **apenas um** `agendamento`, e um `agendamento` pertence a **apenas um** `cliente`.

Relação: Cliente ↔ Ocorrência (Ação: "cliente registra")

- **Leitura do diagrama:** `1..*` (lado cliente) e `1..1` (lado ocorrência).
- **Regra atual:** Um `cliente` pode registrar **várias** `ocorrencias` (ou reclamações), mas cada `ocorrencia` específica é aberta por **apenas um** `cliente`.

Relação: Usuário ↔ Agendamento (Ação: "admin confirma")

- **Leitura do diagrama:** `1..*` (lado usuário) e `..1` (lado agendamento).
- **Regra atual:** Um `usuario` (funcionário/admin) pode confirmar ou gerenciar **vários** `agendamentos`, mas cada `agendamento` é confirmado por **apenas um**

`usuario` .

Relação: Usuário ↔ Ocorrência (Ações: "registra" e "responde")

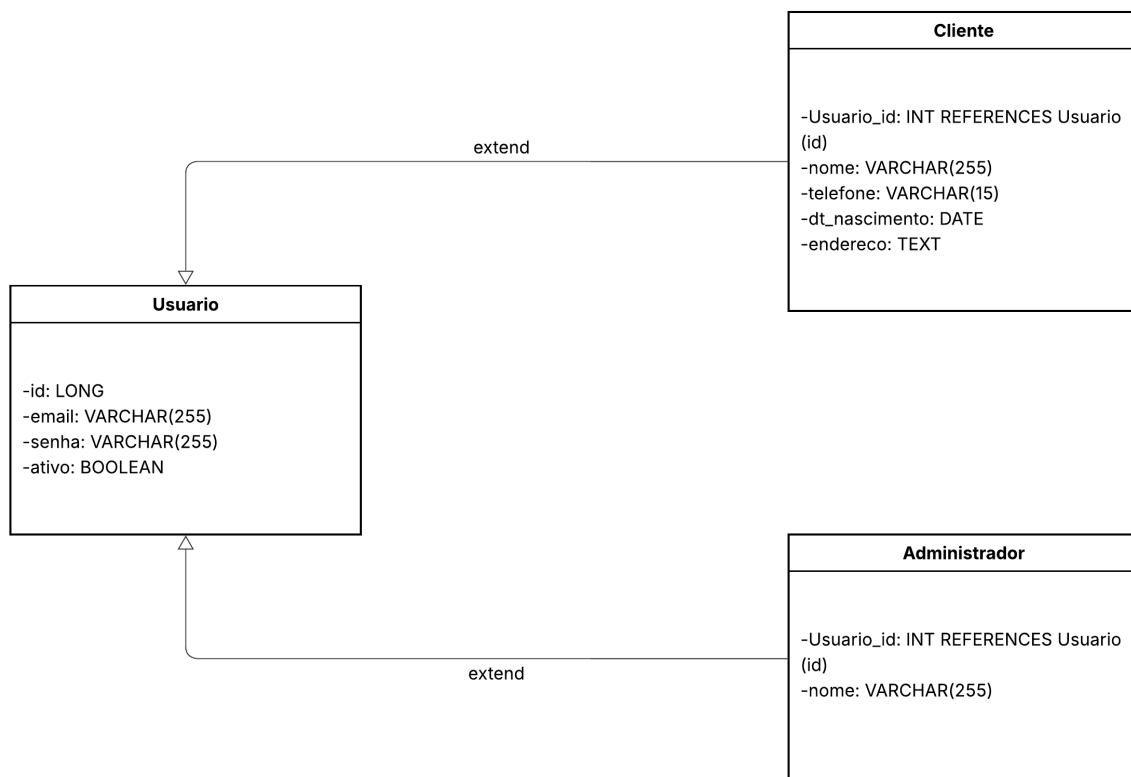
- **Regra (Registra):** Um `usuario` pode registrar **várias** ocorrências internas, e cada ocorrência interna é registrada por apenas um `usuario` .
- **Regra (Responde):** Um `usuario` pode responder a **várias** ocorrências (aquelas abertas pelos clientes), e cada ocorrência tem a resposta de apenas um `usuario` .

Relação: Usuário ↔ Pedido (Ação: "admin atualiza")

- **Leitura do diagrama:** `1..*` e `..1` .
- **Regra atual:** Um `usuario` (admin) gerencia/atualiza **vários** `pedidos` , mas cada `pedido` é supervisionado por **apenas um** `usuario` .

Diagrama Híbrido (Classe & Entidade-Relacionamento (DER))

Autenticação



💡 Próximos Passos

- [] Definir a tabela de equivalência de Pontos vs. Recompensa com a proprietária.
- [] Desenvolver Wireframes (Protótipos de baixa fidelidade).
- [] Validar o volume de dados para decisão final da arquitetura do Banco de Dados.